# Simulating Shear-Thinning Flow in a Constricted Duct

### from Newton's Second Law to the SIMPLE Method and the Power-Law Model

Andres Henrique de Souza Palomo - 2599691

August 27, 2026

**Abstract**

This document describes the development of a shear-thinning flow simulation in C. It starts from Newton's second law, build up the equations that govern an incompressible fluid, and then show, step by step, how those equations become the finite-volume code. Five milestones were set before hand: the base case, simple boundary conditions, the syringe-like constriction, the SIMPLE (projection) method for mass conservation, and finally the power-law model for shear-thinning viscosity.

## Contents

**1 Introduction** **2**

**2 From Newton's Second Law to the Equations of Motion** **3**

2.1 The continuity equation: Mass must be preserved . . . . . 3

2.2 Momentum conservation: Newton's law for a moving fluid parcel . . . . . 3

2.3 Newtonian fluids and the Navier–Stokes equations . . . . . 4

2.4 Incompressible flow: the equations we actually solve . . . . . 4

**3 The Finite Volume Method** **4**

3.1 Discretization of space into control volumes . . . . . 4

3.2 Velocity update based on the momentum equation . . . . . 5

3.3 Intoduction to the Pressure poisson equation . . . . . 6

3.4 Iterative solution of poisson equation with Jacobi and SOR methods . . . . . 7

3.5 The baseline code . . . . . 7

**4 Boundary conditions - Giving a physical shape** **8**

**5 Syringe geometry and conservation of mass** **9**

**6 The SIMPLE method** **10**

6.1 Introduction of SIMPLE method . . . . . 10

6.2 Deriving the projection step . . . . . 10

6.3 Comparison between legacy and SIMPLE methods . . . . . 11

**7 Shear-thinning fluids and the power-law model** **12**

7.1 The rate-of-strain tensor . . . . . 12

7.2 Governing equations for incompressible non-Newtonian flow . . . . . 13

7.3 Re-deriving the pressure Poisson equation for a non-Newtonian fluid . . . . . 13

7.4 Numerical treatment using a lagged update . . . . . 14

7.5 Code implementation of the shear-thinning viscosity . . . . . 15

**8 Models overview and Results** **16**

8.1 Files and milestones . . . . . 16

8.2 Velocity profiles . . . . . 16

8.3 Flow-rate profiles . . . . . 17

8.4 Apparent viscosity . . . . . 18

**9 Conclusion** **19**

# 1 Introduction

The goal of this project is to simulate how a shear-thinning fluid, such as adhesive, a paste, or blood would flow within an application device. Shear-thinning means that the fluid gets less viscous the faster you shear it. The simulation is built up on five stages:

1. **Base case.** The code develop during lectures: the Navier–Stokes equations, the finite-volume discretizations, and the pressure-Poisson equation that ties velocity and pressure together in a periodic, wrap-around domain.
2. **Boundary conditions.** The flow is restricted by a real and physical shape with an inlet, an outlet, and solid walls.
3. **The syringe geometry.** A strangulation in the duct is set. This case shows that the pressure equation inherited from the base case silently fails to conserve mass once the geometry is no longer a straight channel.
4. **The SIMPLE method.** The broken pressure equation is replaced with a proper, industry-standard method, which couples velocity and pressure (the SIMPLE algorithm) to restore mass conservation.
5. **Shear-thinning viscosity.** The constant viscosity is replaced with a power-law model, the governing equations for a viscosity are re-derived considering the local shear rate, giving the final velocity and viscosity profiles.

Two sources do most of the heavy lifting here. The Newtonian physics and the finite-volume machinery (Sections 2–4) follow the lecture notes by Korzec, *Computational Fluid Dynamics*, Chapter 3 [1], which is also the source of the original baseline code. The non-Newtonian extension (Section 7) follows the derivation in the project's rheology reference, Yadav, *Non-Newtonian Fluids for Industrial Applications* [2].

# 2 From Newton's Second Law to the Equations of Motion

Before writing the solver code it helps to see *where* the equations we are about to discretize actually come from. This section is a short introduction based on the Korzec [1], Sections 3.1-3.3.

## 2.1 The continuity equation: Mass must be preserved

A fluid is really billions of interacting molecules, but tracking each one is both hopeless and unnecessary. Instead the fluid is described as a continuum: a mass density field $\rho(\boldsymbol{x}, t)$ and a velocity field $\mathbf{v}(\boldsymbol{x}, t)$. The mass inside some fixed volume $V$ is

$$m(t) = \int_V \rho(\boldsymbol{x}, t)\, d^3x. \tag{1}$$

Mass can only change if matter flows through the boundary $\partial V$, carried by the current density $\boldsymbol{j} = \rho\mathbf{v}$. Conservation of mass therefore reads

$$\frac{d}{dt}\int_V \rho\, d^3x = -\oint_{\partial V} \boldsymbol{j}\cdot d\boldsymbol{\sigma}. \tag{2}$$

Applying the divergence theorem to the right-hand side, and noting that Eq. (2) must hold for *any* volume $V$, the integrands themselves must be equal. This gives the **continuity equation**

$$\frac{\partial \rho}{\partial t} + \nabla\cdot(\rho\mathbf{v}) = 0. \tag{3}$$

## 2.2 Momentum conservation: Newton's law for a moving fluid parcel

The second equation comes from Newton's second law, $m\, d\mathbf{v}/dt = \boldsymbol{F}$, applied to an infinitesimally small parcel of fluid that moves with the flow. Because the parcel's position $\boldsymbol{x}(t)$ is itself changing, differentiating $\mathbf{v}(\boldsymbol{x}(t), t)$ with the chain rule produces both a local (in time) and an advective (in space) piece:

$$\rho\,\frac{D\mathbf{v}}{Dt} \equiv \rho\left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v}\cdot\nabla\mathbf{v}\right) = \boldsymbol{b}, \tag{4}$$

where $D/Dt$ is the *material derivative* and $\boldsymbol{b}$ is the force per unit volume acting on the parcel. That force splits into an external part (gravity, $\boldsymbol{f} = \rho\boldsymbol{g}$) and an internal part coming from how the fluid's own molecules push on each other, which is summarized by the **stress tensor** $\sigma$ :

$$b = \boldsymbol{f} + \nabla\cdot\sigma, \tag{5}$$

Separating $\sigma$ into an isotropic pressure part and a deviatoric (shape-changing) part $\tau$,

$$\sigma = -p\,\mathbb{1} + \tau, \tag{6}$$

gives the most general equations of fluid dynamics, valid for any fluid:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0, \tag{7}$$

$$\rho \frac{D\mathbf{v}}{Dt} = -\nabla p + \nabla \cdot \tau + \rho \boldsymbol{g}. \tag{8}$$

Equation (8) is the **Cauchy momentum equation**. It is still incomplete: we need a *constitutive relation* that tells us how $\tau$ depends on the flow. That relation is exactly what distinguishes a Newtonian fluid (Section 2.3) from a shear-thinning one (Section 7).

### 2.3 Newtonian fluids and the Navier–Stokes equations

For a Newtonian fluid, $\tau$ is required to be isotropic, so there is no tension when fluid is at rest. Also it must be *linearly* depend on the velocity gradient. Under these assumptions it becomes:

$$\tau_{ij} = \mu \left( \frac{\partial v_i}{\partial x_j} + \frac{\partial v_j}{\partial x_i} \right) + \lambda \, \nabla \cdot \mathbf{v}, \tag{9}$$

with $\mu$ the (constant) dynamic viscosity, and $\lambda$ is a second coefficient of viscosity (sometimes called bulk). The constant $\lambda$ only matters when $\nabla \cdot \mathbf{v} \neq 0$, and for a incompressible flow, so it won't be of relevance for this study. The section 2.4 drops it out of the equations entirely. Substituting Eq. (9) into Eq. (8) gives the Navier-Stokes equations

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0, \tag{10}$$

$$\rho \frac{D\mathbf{v}}{Dt} = -\nabla p + \nabla \cdot \left[ \mu \left( \nabla \mathbf{v} + \nabla \mathbf{v}^{\top} \right) \right] + \nabla [\lambda \, \nabla \cdot \mathbf{v}] + \rho \boldsymbol{g}. \tag{11}$$

### 2.4 Incompressible flow: the equations we actually solve

A liquid is considered to be incompressible. So a small parcel of fluid keeps the same density as it moves with the flow, and therefore $d\rho/dt = 0$. Combined with the continuity equation (3), and using the fact that $\rho \neq 0$, this forces the velocity field to be **divergence-free**,

$$\nabla \cdot \mathbf{v} = 0. \tag{12}$$

With a constant and uniform density $\rho_0$, the equations we solve numerically for a Newtonian fluid are:

$$\nabla \cdot \mathbf{v} = 0, \tag{13}$$

$$\rho_0 \frac{D\mathbf{v}}{Dt} = -\nabla p + \mu \, \Delta \mathbf{v} + \rho_0 \boldsymbol{g}. \tag{14}$$

This equation is the governing rule to develop the Finite Volume Method for Newtonian fluids.

## 3 The Finite Volume Method

### 3.1 Discretization of space into control volumes

To solve Eqs. (13)–(14) numerically we tile the domain into small rectangular cells (in 2D: cells of size $\Delta x \times \Delta y$). Each cell has a **control point** $\boldsymbol{c}$ in its center, and this is where every field value ($p$, $v_x$,

$v_y$) is stored. Every cell has four neighbors, denoted East, West, north, and South ($E, W, N, S$), and the center of the face shared with each neighbor is denoted $\boldsymbol{e}, \boldsymbol{w}, \boldsymbol{n}, \boldsymbol{s}$ [1]. This is exactly the data structure at the top of every C file in this project:

Listing 1: The core arrays of the baseline solver (`finite_vol_1_baseline.c`). Every field lives on the same $N_x \times N_y$ grid of control points.

```c
double p[Nx*Ny]; // pressure in control points
double vx[Nx*Ny]; // x-component of the velocity in control points
double vy[Nx*Ny]; // y-component of the velocity in control points
int E[Nx*Ny], W[Nx*Ny], N[Nx*Ny], S[Nx*Ny]; // neighbour indices
```

The function `site2index` folds a 2D coordinate $(x, y)$ into the 1D array index used to store every field, and `init_control_points` pre-computes the $E, W, N, S$ index of every cell once, so the rest of the solver never has to repeat that bookkeeping.

### 3.2 Velocity update based on the momentum equation

Integrating the momentum equation over one control volume $V$ and applying the divergence theorem turns every volume integral of a divergence into a surface integral over the six faces of the cell [1]. Restricting to 2D, using the lowest-order (midpoint) approximation for every integral, and dividing through by the cell volume $\Delta x\, \Delta y$, component $k \in \{x, y\}$ of the momentum equation becomes the semi-discrete statement

$$\begin{aligned} \frac{v_k(\boldsymbol{c}, t+\Delta t) - v_k(\boldsymbol{c}, t)}{\Delta t} &+ v_x \partial_x v_k(\boldsymbol{c}) + v_y \partial_y v_k(\boldsymbol{c}) \\ &= -\frac{1}{\rho_0}\Big[p(\boldsymbol{e}) - p(\boldsymbol{w})\Big]\frac{\delta_{kx}}{\Delta x} - \frac{1}{\rho_0}\Big[p(\boldsymbol{n}) - p(\boldsymbol{s})\Big]\frac{\delta_{ky}}{\Delta y} \\ &\quad + \frac{\mu}{\rho_0}\Big(\partial_y v_k(\boldsymbol{n}) - \partial_y v_k(\boldsymbol{s})\Big) + \frac{\mu}{\rho_0}\Big(\partial_x v_k(\boldsymbol{e}) - \partial_x v_k(\boldsymbol{w})\Big) + g_k. \end{aligned} \tag{15}$$

Nothing new has been assumed physically here: Eq. (15) is exactly the incompressible, constant-density momentum equation, Eq. (14) (which is in turn Eq. (11) once $\lambda$ drops out and density is frozen at $\rho_0$), just rewritten component-by-component as a balance between what is stored at the control point $\boldsymbol{c}$ and what is evaluated at its four surrounding faces. It is the same equation, in the notation the discretizations actually need. Before turning this into something a computer can evaluate, it helps to fix the notation for "where" each quantity lives. Lowercase bold letters, $\boldsymbol{c}, \boldsymbol{e}, \boldsymbol{w}, \boldsymbol{n}, \boldsymbol{s}$, denote *locations*: the control point at the center of the reference cell, and the four face centers on its boundary, where face values are obtained by interpolation (Section 3.5). Uppercase letters, $E, W, N, S$, instead denote the *control point*, the center, of the corresponding neighbouring cell, i.e. $E$ is not a location on the shared face but the center of the cell to the east. The remaining spatial derivatives are approximated with the simplest central differences consistent with the leading-order scheme:

$$\partial_x v_k(\boldsymbol{c}) = \frac{v_k(\boldsymbol{e}) - v_k(\boldsymbol{w})}{\Delta x}, \qquad \partial_y v_k(\boldsymbol{c}) = \frac{v_k(\boldsymbol{n}) - v_k(\boldsymbol{s})}{\Delta y}, \tag{16}$$

$$\partial_x v_k(\boldsymbol{e}) = \frac{v_k(E) - v_k(\boldsymbol{c})}{\|E - \boldsymbol{c}\|}, \qquad \partial_x v_k(\boldsymbol{w}) = \frac{v_k(\boldsymbol{c}) - v_k(W)}{\|\boldsymbol{c} - W\|}, \tag{17}$$

$$\partial_y v_k(\boldsymbol{n}) = \frac{v_k(N) - v_k(\boldsymbol{c})}{\|N - \boldsymbol{c}\|}, \qquad \partial_y v_k(\boldsymbol{s}) = \frac{v_k(\boldsymbol{c}) - v_k(S)}{\|\boldsymbol{c} - S\|}. \tag{18}$$

These are, respectively, Eqs. 3.53–3.58 in Korzec's notes, and the same numbers appear as comments directly above the corresponding lines in `update_velocity()` in the code. Two of these face derivatives always appear as a pair, one minus the other, so it is worth naming that combination on its own before it gets used:

$$\partial_x^{ew} v_k \;\equiv\; \partial_x v_k(\boldsymbol{e}) - \partial_x v_k(\boldsymbol{w}), \qquad \partial_y^{ns} v_k \;\equiv\; \partial_y v_k(\boldsymbol{n}) - \partial_y v_k(\boldsymbol{s}). \tag{19}$$

That is the finite-volume stencil for a second derivative, so $\partial_x^{ew} v_k + \partial_y^{ns} v_k$ is the discrete Laplacian $\Delta v_k$. Putting everything together gives the explicit time-marching update used everywhere in this project (Korzec Eq. 3.59):

$$v_k(\boldsymbol{c}, t+\Delta t) = v_k(\boldsymbol{c}, t) + \Delta t\Big[ - v_x\, \partial_x v_k - v_y\, \partial_y v_k - \tfrac{1}{\rho_0}\nabla_k p + \tfrac{\mu}{\rho_0}\big(\partial_x^{ew} v_k + \partial_y^{ns} v_k\big) + g_k\Big], \tag{20}$$

where $\nabla_k p$ stands for the appropriate face-to-face pressure difference and $\partial_x^{ew} v_k$, $\partial_y^{ns} v_k$ are the discrete-Laplacian terms just defined in Eq. (19), i.e. the viscous diffusion piece of the momentum equation. This is a completely explicit scheme: every quantity on the right-hand side is known at time $t$, so no linear system has to be solved for the velocity update itself.

### 3.3 Intoduction to the Pressure poisson equation

Explicit time-marching gives us new velocities, but nothing so far has enforced the condition $\nabla \cdot \mathbf{v} = 0$. The trick, standard in incompressible-flow solvers, is to take the divergence of the momentum equation and solve the result for pressure. The steps are:

**Step 1.** Start from Eq. (14) in constant-density form,

$$\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v}\cdot\nabla)\mathbf{v} = -\frac{1}{\rho_0}\nabla p + \frac{\mu}{\rho_0}\Delta\mathbf{v} + \boldsymbol{g}, \tag{21}$$

**Step 2.** Isolate the pressure gradient,

$$\nabla p = -\rho_0 \frac{\partial \mathbf{v}}{\partial t} - \rho_0\,(\mathbf{v}\cdot\nabla)\mathbf{v} + \mu\Delta\mathbf{v} + \rho_0\boldsymbol{g}. \tag{22}$$

**Step 3.** Take the divergence of both sides,

$$\nabla\cdot(\nabla p) = \nabla\cdot\left[-\rho_0 \frac{\partial \mathbf{v}}{\partial t} - \rho_0\,(\mathbf{v}\cdot\nabla)\mathbf{v} + \mu\Delta\mathbf{v} + \rho_0\boldsymbol{g}\right]. \tag{23}$$

**Step 4.** Rewrite the divergence of the gradient as the Laplacian,

$$\Delta p = \nabla\cdot\left[-\rho_0 \frac{\partial \mathbf{v}}{\partial t} - \rho_0\,(\mathbf{v}\cdot\nabla)\mathbf{v} + \mu\Delta\mathbf{v} + \rho_0\boldsymbol{g}\right]. \tag{24}$$

**Step 5.** Enforce incompressibility. Two terms vanish here, for two different reasons. Since $\nabla\cdot(\partial\mathbf{v}/\partial t) = \partial_t(\nabla\cdot\mathbf{v})$ and $\nabla\cdot\mathbf{v} = 0$ at every instant, the time-derivative term vanishes identically, as it will again in Section 7.3. But the viscous term vanishes too, and for a reason specific to this Newtonian, constant-$\mu$ case: because $\mu$ is a single number rather than a field, it can be pulled outside the divergence, so $\nabla\cdot(\mu\Delta\mathbf{v}) = \mu\Delta(\nabla\cdot\mathbf{v}) = 0$. Both terms drop out together, leaving a **Poisson equation for the pressure**:

$$\Delta p = \rho_0\,\nabla\cdot\big[\boldsymbol{g} - (\mathbf{v}\cdot\nabla)\mathbf{v}\big]. \tag{25}$$

For incompressible flow the advective term simplifies further. Writing it out in 2D and using $\partial_x v_x = -\partial_y v_y$ (which follows directly from $\nabla \cdot \mathbf{v} = 0$) collapses several second-derivative terms:

$$\nabla \cdot \big[(\mathbf{v} \cdot \nabla)\mathbf{v}\big] = (\partial_x v_x)^2 + (\partial_y v_y)^2 + 2(\partial_x v_y)(\partial_y v_x), \tag{26}$$

so the equation actually solved is

$$\Delta p = \rho_0 \, \nabla \cdot \boldsymbol{g} - \rho_0 \Big[(\partial_x v_x)^2 + (\partial_y v_y)^2 + 2(\partial_x v_y)(\partial_y v_x)\Big]. \tag{27}$$

This is Korzec Eqs. 3.60–3.62. Note what Eq. (27) actually is: a Poisson equation whose right-hand side is built assuming $\nabla \cdot \mathbf{v} = 0$ *already holds*. It never measures whether the velocity field it is given actually satisfies continuity, but only asks "what pressure field would keep a genuinely divergence-free flow consistent with the momentum equation." That difference seems irrelevant when it's on a normal channel, but it turns out to of high impact when the geometry changes (Section 5).

### 3.4 Iterative solution of poisson equation with Jacobi and SOR methods

Equation (27) is discretized on the same 5-point stencil as before,

$$\frac{-2p(\boldsymbol{c}) + p(\boldsymbol{n}) + p(\boldsymbol{s})}{(\Delta y)^2} + \frac{-2p(\boldsymbol{c}) + p(\boldsymbol{e}) + p(\boldsymbol{w})}{(\Delta x)^2} = b(\boldsymbol{c}), \tag{28}$$

where $b(\boldsymbol{c})$ is the right-hand side of Eq. (27) evaluated at the current velocity field. Unlike the momentum update, this is a *linear system that couples every cell to its neighbours simultaneously*,b setting $p$ at one cell to satisfy Eq. (28) perturbs the equation at every neighbouring cell. Korzec's notes (Section 3.5.5) resolve this with Jacobi iteration: freeze the source term, then repeatedly sweep over every cell updating

$$p^{\text{new}}(\boldsymbol{c}) = \frac{-b(\boldsymbol{c})\,\Delta x^2 \Delta y^2/4 + \big(p(\boldsymbol{n}) + p(\boldsymbol{s})\big)\Delta x^2 + \big(p(\boldsymbol{e}) + p(\boldsymbol{w})\big)\Delta y^2}{2\big(\Delta x^2 + \Delta y^2\big)} \tag{29}$$

until $\|p^{\text{new}} - p\|$ drops below a tolerance. This project also later uses the SOR method instead of Jacobi (from `finite_vol_2_duct.c` onward), since it allows for faster convergence.

$$p_i \leftarrow (1-\omega)\, p_i + \omega\, p_i^{\text{Gauss–Seidel}} \tag{30}$$

with the relaxation factor set to $\omega = 1.7$ in the code. Gauss–Seidel (using already-updated neighbour values within the same sweep).

### 3.5 The baseline code

`finite_vol_1_baseline.c` implements the pipeline above: `setup()` builds the grid and neighbour lists, `interpolate_faces()` averages cell-center values onto the four faces, `update_velocity()` applies Eq. (20), and `update_pressure()` applies one Jacobi-style sweep, Eq. (29). It is deliberately left as a *base case*: milestone 1 of the project is understanding the physics and the code structure, not fixing anything yet. Two features of this baseline are worth flagging early, because they are what the next two sections replace:

- **The domain is periodic.** `site2index` wraps coordinates with the modulo operator, so the grid is really a torus with no true walls, inlet or outlet. Section 4 replaces this with a real bounded geometry.
- **The pressure equation is applied once per step, not solved to convergence.** `update_pressure()` performs a single relaxation-style update per outer iteration rather than iterating Eq. (29) (or its SOR variant) until it converges. Section 4 fixes this by nesting a converged SOR solve inside every outer step, and Section 6 goes further, replacing the underlying formula itself once the geometry stops being a simple channel.

## 4 Boundary conditions - Giving a physical shape

A well-posed problem needs an initial condition and a boundary condition on $\partial\Omega$, $\mathbf{v}(\boldsymbol{x}, t) = \mathbf{v}_b(\boldsymbol{x}, t)$ for $\boldsymbol{x} \in \partial\Omega$ [1]. Milestone 2 (`finite_vol_2_duct.c`) gives the domain that physical shape using the standard **ghost-cell** technique: the physical $N_x \times N_y$ interior is surrounded by one extra ring of cells, so the full array is $(N_x + 2) \times (N_y + 2)$. The ghost ring never represents real fluid; instead its values are set, every step, so that interpolating a face value between a ghost cell and its physical neighbour reproduces the boundary condition we actually want:

- **Inlet ($x = 0$), Dirichlet velocity.** $v_x = U_{\text{in}}$, $v_y = 0$ in the ghost column, together with zero-gradient (Neumann) pressure, $p_{\text{ghost}} = p_{\text{first interior cell}}$.
- **Outlet ($x = N_x + 1$), Dirichlet pressure.** $p = P_{\text{out}}$ in the ghost column, together with zero-gradient velocity, $v_{\text{ghost}} = v_{\text{last interior cell}}$, since fluid is free to leave without the outlet forcing a particular exit profile.
- **Top and bottom walls, no-slip.** A no-slip condition, $\mathbf{v}_b = 0$ [1], is enforced not by setting the wall itself to zero (there is no control point exactly on the wall) but by *mirroring*: $v_{\text{ghost}} = -v_{\text{first interior cell}}$, so that the face-averaged velocity used by the solver, $\frac{1}{2}(v_{\text{ghost}} + v_{\text{interior}})$, is zero right on the wall.

Listing 2: No-slip wall implemented via a mirrored ghost value (`finite_vol_2_duct.c`).

```
int g=site2index(x,0);
int in1=site2index(x,1);
vx[g]=-vx[in1];
vy[g]=-vy[in1];
```

The other change in this milestone is solving Eq. (28) *properly*: `solve_pressure_sor()` freezes the source term $b_{\text{src}}$ once per outer step and then iterates the SOR update, Eq. (30), until the residual falls below an inner tolerance (or a sweep cap is hit), rather than the single Jacobi-style pass of the baseline code. Because the ghost pressure values feed back into the face pressures of boundary-adjacent cells, `apply_boundary_conditions()` is re-applied after every SOR sweep. With a real inlet, outlet and walls, and a converged pressure solve, this is the first version of the code where a proper wall-to-wall parabolic (Poiseuille-like) velocity profile can develop.

# 5 Syringe geometry and conservation of mass

Milestone 3 introduces a syringe-like constriction in the duct to represent the geometry of adhesive application more realistically. The channel narrows to half of its original height, like the nozzle of an adhesive applicator. A solid cell is simply an ordinary grid cell that is excluded from the momentum and pressure updates and held at $\mathbf{v} = 0$ with no-slip boundary conditions applied on its faces:

Listing 3: The contraction geometry: everything downstream of `CONTRACTION_X` is solid except a central band of height $N_y/2$ (`finite_vol_3_syringe.c`).

```
#define CONTRACTION_X 50
#define THROAT_Y_LO (Ny/4+1)  // first open row of the throat
#define THROAT_Y_HI (3*Ny/4)  // last open row of the throat
for (int x=1;x<=Nx;x++)
  for (int y=1;y<=Ny;y++)
    if (x>CONTRACTION_X && (y<THROAT_Y_LO || y>THROAT_Y_HI))
      solid[i]=1;
```

With $N_y = 20$ this leaves a throat 10 cells tall out of 20, half the tube height, centered on the duct's mid-line, giving a 2:1 area ratio between the wide section and the throat. Pressure boundary conditions on solid cells are handled with a zero-gradient (neighbour-average) rule, so the pressure field stays smooth right up against the constriction wall. In this implementation phase, the necessity to review the previous approach ended up to be necessary. Without it, simulating anything else, other than a simple duct would too far from the reality. The pressure equation derived in Section 3.3, Eq. (27), was obtained by *assuming* $\nabla \cdot \mathbf{v} = 0$ already holds and asking what pressure field is then consistent with the momentum equation. The source term $b_{\text{src}}$ in Eq. (28) is built from whatever velocity field the solver currently has, but it never *measures* the actual mass imbalance and corrects for it. In a straight channel it can be overlooked, since the flow has nowhere else to go. It becomes a real problem once a change in cross-section forces the flow rate to different directions. When the syringe shape is introduced, using the the baseline code, the simulation failed and flow simply disappeared . How badly this fails can be seen with a direct numerical check. The volumetric flow rate through a cross-section at axial position $x$ is defined as

$$Q(x) = \sum_y v_x(x, y)\, \Delta y. \tag{31}$$

Mass conservation requires $Q(x)$ to be constant along the whole duct, equal to $U_{\text{in}} N_y \approx 2.0$ in these general units. Running `finite_vol_2_duct.c` (legacy pressure, plain duct, *no* constriction at all) to full convergence and evaluating $Q(x)$ near the inlet and outlet already gives a **58%** mismatch between the two, and the maximum cell-wise divergence $\max |\nabla \cdot \mathbf{v}|$ does not shrink with further iterations, instead converging to a steady, nonzero value of about $3.9 \times 10^{-2}$ (see Table 1, row 1). Adding the constriction, `finite_vol_3_syringe.c`, makes it even worse. The mismatch grows to **85%** (Table 1, row 2, and Figure 1). So the geometry is not what breaks mass conservation, it is what makes pressure treatment impossible to ignore. This is what Section 6 fixes.

# 6 The SIMPLE method

## 6.1 Introduction of SIMPLE method

The fix is to stop assuming continuity and start *enforcing* it. This is the idea behind the **SIMPLE** algorithm (Semi-Implicit Method for Pressure-Linked Equations): the velocity is advanced with a *guessed* or *provisional* pressure field, how badly the resulting velocity field violates $\nabla \cdot \mathbf{v} = 0$ is measured, and that measured imbalance is used to build a pressure correction that restores continuity. Milestone 4 implements this as an explicit, fully time-marching scheme, following the same core structure of predicting, measuring the imbalance, and correcting. SIMPLE is an industry-standard method in computational fluid dynamics, described in detail in Baheri and Navaserarab [3], and the implementation here is validated directly against the mass-conservation numbers (Section 6.3).

## 6.2 Deriving the projection step

The momentum update, Eq. (20), is split into two pieces. First the velocity is advanced using only advection and viscous diffusion, with no pressure term at all, to get a **predicted velocity** $\mathbf{v}^*$:

$$\mathbf{v}^* = \mathbf{v}^n + \Delta t \Big[ -(\mathbf{v}^n \cdot \nabla)\mathbf{v}^n + \tfrac{1}{\rho_0} \nabla \cdot \tau^n \Big]. \tag{32}$$

There is no reason for $\mathbf{v}^*$ to be divergence-free; in general it will not be. What is sought next is the pressure field $\phi$ that, when subtracted off as a gradient correction, restores continuity in the corrected velocity:

$$\mathbf{v}^{n+1} = \mathbf{v}^* - \Delta t \, \nabla \phi, \qquad \text{such that } \nabla \cdot \mathbf{v}^{n+1} = 0. \tag{33}$$

Taking the divergence of Eq. (33) and imposing $\nabla \cdot \mathbf{v}^{n+1} = 0$ gives a Poisson equation, but this time for the pressure *correction* $\phi$, and driven by the *actual*, measured divergence of the predicted field:

$$\boxed{\Delta \phi = \frac{\nabla \cdot \mathbf{v}^*}{\Delta t}}. \tag{34}$$

This is the crucial difference from Eq. (27): the right-hand side is not derived by assuming continuity, it is computed directly from a velocity field that generally violates continuity, so it is the correction needed to cancel that violation. Equation (34) is discretized on the same 5-point stencil and solved with the same SOR machinery as before (Eqs. (28)–(30)); only the source term has changed. Once $\phi$ has converged, the velocity is corrected with Eq. (33), and $\phi$ is retained as (an update to) the pressure field, ready for the next time step.

Listing 4: The four steps of the projection method as implemented in `finite_vol_4_syringe_SIMPLE.c`: predict, measure the imbalance, solve for the correction, correct.

```
predict_velocity();            // Eq. (29): vxstar,vystar -- no pressure term
compute_b_src();               // Eq. (31): b_src = div(v*)/dt
solve_pressure_sor(...);       // solve Delta(phi) = b_src via SOR
correct_velocity();            // vx = vxstar - dt*dphi/dx (and vy analogously)
```

It is worth being precise about naming: what is implemented here is, strictly, an explicit, fully time-marching projection method, rather than an implicit, steady-state, under-relaxed formulation. The project refers to it as "the SIMPLE method" because it belongs to the same family and solves

the same problem the same way, by correcting a provisional velocity using a pressure computed from the *actual* mass imbalance, instead of one derived by assuming the imbalance is already zero, and the code comments describe it accordingly as "the SIMPLE algorithm's explicit-time-marching sibling."

### 6.3 Comparison between legacy and SIMPLE methods

Three source files make this comparison, sharing the same $100 \times 20$ grid and each run to full convergence: `finite_vol_2_duct.c` (plain duct, legacy pressure), `finite_vol_3_syringe.c` (same duct, with the constriction added), and `finite_vol_4_syringe_SIMPLE.c` (same constriction, legacy pressure replaced by SIMPLE). Table 1 and Figure 1 report the converged result of each.

Table 1: Converged flow-rate mismatch for each milestone file.

| File | Geometry / viscosity / pressure | $\max \lvert \nabla \cdot \mathbf{v} \rvert$ | $Q_{\text{in}}$ | $Q_{\text{out}}$ | Mismatch |
|---|---|---|---|---|---|
| `finite_vol_2_duct` | duct, Newtonian, legacy | $3.9 \times 10^{-2}$ | 1.673 | 0.704 | **57.9%** |
| `finite_vol_3_syringe` | syringe, Newtonian, legacy | $3.9 \times 10^{-2}$ | 1.671 | 0.244 | **85.4%** |
| `finite_vol_4_syringe_SIMPLE` | syringe, Newtonian, SIMPLE | $4.8 \times 10^{-4}$ | 1.996 | 2.031 | **1.7%** |

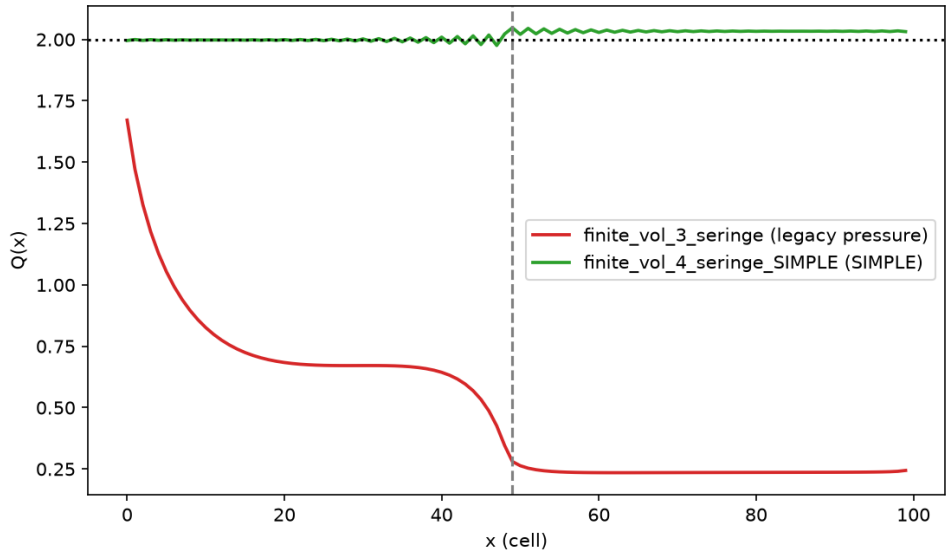


Figure 1: Flow rate $Q(x)$ along the duct: legacy pressure (red) versus SIMPLE (green).

The first two rows repeat the finding from Section 5: the legacy pressure formula already loses most of the flow rate on a plain duct (row 1), and the constriction makes it worse (row 2). Neither row's $\max \lvert \nabla \cdot \mathbf{v} \rvert$ moves much from the other, confirming that it is the pressure formula, not the geometry, that is at fault. It might look strange that $Q_{\text{in}}$ itself, not only $Q_{\text{out}}$, already sits well below the nominal $U_{\text{in}} N_y = 2.0$ in these first two rows, since $U_{\text{in}}$ is fixed by a boundary condition. But the Dirichlet condition only fixes the velocity in the ghost column outside the domain; $Q_{\text{in}}$ is measured one cell inside it, at a control point the momentum equation is free to solve for. Because Eq. (27) is solved as one coupled system over the whole grid, a source term that is wrong everywhere corrupts

the solution starting from that very first interior column, not only far downstream. Row 3 is what SIMPLE (Section 6) buys back. With the identical syringe geometry, switching only the pressure algorithm cuts the mismatch from 85% to 1.7% and the maximum divergence by nearly two orders of magnitude. The residual few percent is ordinary discretization and SOR convergence error, not a modeling failure. Figure 1 shows the full $Q(x)$ profile along the duct for the syringe geometry, legacy pressure versus SIMPLE: the legacy formula lets $Q(x)$ drift and eventually crash through the constriction, while SIMPLE keeps it essentially flat across the whole domain, including right through the throat.

# 7 Shear-thinning fluids and the power-law model

With the mechanical bookkeeping (grid, boundary conditions, mass-conserving pressure coupling) in place, milestone 5 replaces the constant Newtonian viscosity $\mu$ with something that responds to the flow itself.

A Newtonian fluid has vicosity constant, but many real fluids do not behave that way, like adhesives, paints or blood, which all get less resistant to flow the faster they are locally sheared, a behaviour called shear-thinning.

The simplest constitutive model that captures this is the **power law** model [2], which lets the apparent viscosity fall off as a power of the local shear rate $\dot{\gamma}$,

$$\eta_a = K\,\dot{\gamma}^{\,n-1}, \tag{35}$$

with a consistency index $K > 0$ and a flow behaviour index $n$. Three regimes fall out of the exponent: at $n = 1$, Eq. (35) collapses back to a constant viscosity, $\eta_a = K = \mu$, recovering the Newtonian case exactly. For $n < 1$ the viscosity *decreases* as the shear rate grows, which is the shear-thinning behaviour of this project. For $n > 1$ it would be the opposit, like cornstarch suspension. For this study, $K$ **and** $n$ **were set to** 1 **and** 0.5, respectively.

To turn Eq. (35) into something usable inside a momentum equation, two things are still missing: what tensor quantity $\dot{\gamma}$ actually is for a 2D velocity field, and how the resulting $\eta_a$ feeds back into the stress. Both come from, and the rest of this section follows step by step, the reference book's rheology chapter, Section 2.3.5, *Power Law Fluids* [2].

## 7.1 The rate-of-strain tensor

The kinematic link between the velocity field and the internal viscous stresses is the **rate-of-strain tensor D**, the symmetric part of the velocity gradient:

$$D_{ij} = \frac{1}{2}\left(\frac{\partial v_i}{\partial x_j} + \frac{\partial v_j}{\partial x_i}\right). \tag{36}$$

For a generalized (non-Newtonian) Newtonian-*like* fluid, the stress is still taken proportional to **D**, but now through the apparent viscosity $\eta_a$ of Eq. (35), which is itself a function of the local flow state:

$$\tau = 2\,\eta_a\,\mathbf{D}. \tag{37}$$

To turn the tensor **D** into the single scalar number that Eq. (35) actually needs, its second invariant is used: the **shear rate**

$$\dot{\gamma} = \sqrt{2\,\mathbf{D} : \mathbf{D}}, \tag{38}$$

where $\mathbf{D} : \mathbf{D} = \sum_{i,j} D_{ij} D_{ij}$ is the tensor double-dot (Frobenius) product. This scalar magnitude is coordinate-system independent, which is what makes it a sensible input to Eq. (35). *From tensor to code.* Written out for the 2D grid, $\mathbf{D} : \mathbf{D} = D_{xx}^2 + D_{yy}^2 + 2D_{xy}^2$ (the cross term counts twice, since $D_{xy} = D_{yx}$), so Eq. (38) becomes $\dot{\gamma} = \sqrt{2\left(D_{xx}^2 + D_{yy}^2 + 2D_{xy}^2\right)}$. That is what `compute_viscosity()` does. The code computes $\dot{\gamma}$ itself, so $K$ and $n$ can be compared directly against real viscometer data reported in terms of $\dot{\gamma}$ with no conversion factor to keep track of. Since there are many indices, Table 2 compares the formulas presented above with the code, to make it easier to read.

Table 2: Report notation versus code variables in `compute_viscosity()`.

| Report (Eq. (36)) | Code | Meaning |
|---|---|---|
| $D_{xx} = \partial v_x / \partial x$ | `Dxx = (uxe-uxw)/dx` | normal strain rate, $x$-direction |
| $D_{yy} = \partial v_y / \partial y$ | `Dyy = (uyn-uys)/dy` | normal strain rate, $y$-direction |
| $\partial v_x / \partial y$ | `dyvx = (uxn-uxs)/dy` | tangential gradient |
| $\partial v_y / \partial x$ | `dxvy = (uye-uyw)/dx` | tangential gradient |
| $D_{xy} = \frac{1}{2}(\partial v_x / \partial y + \partial v_y / \partial x)$ | `Dxy = 0.5*(dyvx+dxvy)` | shear strain rate |
| $\dot{\gamma} = \sqrt{2\,\mathbf{D} : \mathbf{D}}$ | `gamma_dot = sqrt(2*DdotD)` | shear rate, Eq. (38) |

### 7.2 Governing equations for incompressible non-Newtonian flow

With the constitutive relation Eq. (37) in hand, the governing system for an incompressible non-Newtonian fluid is

$$\nabla \cdot \mathbf{v} = 0, \tag{39}$$

$$\rho \left( \frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \right) = -\nabla p + \nabla \cdot (2\,\eta_a\,\mathbf{D}) + \boldsymbol{F}. \tag{40}$$

The crucial difference from the Newtonian case, Eq. (14), is that $\eta_a$ now depends on $\mathbf{D}$, which depends on $\mathbf{v}$. This is a genuine feedback loop: the velocity gradient sets $\dot{\gamma}$, which sets $\eta_a$, which changes how momentum diffuses, which changes the velocity gradient. It also means the simplification used to reduce $\nabla \cdot (\mu \nabla \mathbf{v} + \mu \nabla \mathbf{v}^\top)$ down to $\mu \Delta \mathbf{v}$ in Section 2.4 *no longer applies*: because $\eta_a$ varies from cell to cell, it cannot be pulled out of the divergence, so the viscous term must be kept in its full stress-divergence form, $\nabla \cdot (2\eta_a \mathbf{D})$, everywhere.

### 7.3 Re-deriving the pressure Poisson equation for a non-Newtonian fluid

Because the viscous term no longer collapses to a Laplacian, the pressure-Poisson derivation has to be redone starting from the generalized (Cauchy) momentum equation, Eq. (8), rather than the Newtonian Navier–Stokes form. The algebra, however, is exactly the same five steps already carried out in Section 3.3: expand the material derivative, isolate $\nabla p$, take the divergence of both sides, rewrite $\nabla \cdot \nabla p$ as $\Delta p$, then enforce incompressibility. But this time the second cancellation, Step 5 of Section 3.3, does not happen: $\eta_a$ is a field, not a constant, so it cannot be pulled outside the divergence, and $\nabla \cdot (\nabla \cdot \tau)$ has to be carried through to the end. What remains is the **pressure Poisson equation for a generalized non-Newtonian fluid**:

$$\boxed{\Delta p = -\rho\, \nabla \cdot \left[ (\mathbf{v} \cdot \nabla) \mathbf{v} \right] + \nabla \cdot (\nabla \cdot \tau) + \nabla \cdot \boldsymbol{F}}. \tag{41}$$

Two consequences follow directly from Eq. (41):

- The extra term $\nabla \cdot (\nabla \cdot \tau)$ does not vanish here the way it did in the Newtonian case (Eq. (27)). Because $\tau = 2\eta_a \mathbf{D}$ depends non-linearly on $\mathbf{v}$ through $\eta_a(\dot{\gamma})$, a solver built on Eq. (41) would have to rebuild that term from the current $\mathbf{v}$ and $\eta_a$ every single iteration.
- The projection method (Section 6) never actually needs Eq. (41). Its pressure source, Eq. (34), is the *measured* divergence of the predicted velocity $\mathbf{v}^*$, whatever stress model produced it, so `finite_vol_5_duct_SIMPLE_shearthinning.c` and `finite_vol_6_syringe_SIMPLE_shearthinning.c` reuse the identical Newtonian pressure solve of Section 6, with no viscosity-dependent term added anywhere.

What neither of the paths avoid is the circularity already sitting inside the momentum equation itself: $\eta_a$ depends on $\mathbf{D}(\mathbf{v})$, which depends on the very $\mathbf{v}$ being solved for, so $\mathbf{v}$ and $p$ cannot be written down as one linear system. To break this deadlock, *lagging* $\eta_a$ one step behind $\mathbf{v}$, similar to what has already been done for the velocity update in Section 6 (freeze one quantity, solve for the other, then update and repeat), is the simplest solution.

### 7.4 Numerical treatment using a lagged update

This outer iteration breaks the circularity above by lagging $\eta_a$ one step behind $\mathbf{v}$, so information only ever flows one way within a single outer step:

$$\mathbf{v}^{(k)} \longrightarrow \mathbf{D}\big(\mathbf{v}^{(k)}\big) \longrightarrow \dot{\gamma}^{(k)} \longrightarrow \eta_a^{(k)} \longrightarrow \mathbf{v}^{(k+1)},$$

with $\eta_a^{(k)}$ frozen for the whole step, so nothing depends on the velocity it is about to help produce. Formally, given the $k$-th velocity iterate $\mathbf{v}^{(k)}$,

$$\eta_a^{(k)} = K\left[D_{\text{eff}}\big(\mathbf{D}(\mathbf{v}^{(k)})\big)\right]^{n-1}, \tag{42}$$

$$\rho\,\frac{D\mathbf{v}^{(k+1)}}{Dt} = -\nabla p^{(k+1)} + \nabla \cdot \big(2\,\eta_a^{(k)}\,\mathbf{D}(\mathbf{v}^{(k+1)})\big) + \boldsymbol{F}, \qquad \nabla \cdot \mathbf{v}^{(k+1)} = 0, \tag{43}$$

i.e. Eq. (42) is a pure post-processing step on the *old* velocity, and Eq. (43) is then an ordinary, linear, constant-(per-cell)-viscosity momentum/pressure solve for the *new* one, using the same machinery as Sections 3–6. One pass through the loop is:

1. Initialize $\eta_a$ with a first guess (the zero-shear regularized value; see below).
2. Advance velocity and pressure with the *current* (lagged) $\eta_a$, Eq. (43).
3. Recompute $\dot{\gamma}$ from the new velocity and update $\eta_a$, Eq. (42).
4. Check $\|\mathbf{v}^{\text{new}} - \mathbf{v}^{\text{old}}\| < \epsilon$; otherwise go back to step 2.

This is the same outer time-marching loop already in the code (Section 6), with step 3 tucked in. Section 7.5 walks through it call by call. A well-known numerical hazard of the power law is its behaviour as $\dot{\gamma} \to 0$: for $n < 1$ (shear-thinning), $\eta_a = K\dot{\gamma}^{\,n-1} \to \infty$, which is physically impossible and will produce an ill-conditioned momentum, for example right at the channel centerline of a

symmetric duct, where the velocity profile is "flat". The code regularizes this by replacing the raw shear-rate invariant with a constrain $\varepsilon$ (`POWERLAW_EPSILON`, set to $10^{-6}$ in the code)

$$D_{\text{eff}} = \sqrt{\dot{\gamma}^2 + \varepsilon^2}, \qquad \eta_a = K\, D_{\text{eff}}^{n-1}, \tag{44}$$

This caps the zero-shear viscosity at a large but finite value, $\eta_0 = K\varepsilon^{n-1}$, instead of letting it diverge.

### 7.5 Code implementation of the shear-thinning viscosity

Section 7.4's four abstract steps are, concretely, this sequence of calls inside `solver()`'s outer loop (`finite_vol_5_duct_SIMPLE_shearthinning.c` and `finite_vol_6_syringe_SIMPLE_shearthinning.c`), threaded through the SIMPLE machinery of Section 6:

1. `apply_velocity_bc()` and `interp_v_faces()` refresh the ghost ring and face values of the *current* velocity $\mathbf{v}^{(k)}$ (on the syringe geometry, `apply_solid_bc_v()` also re-applies no-slip at the constriction wall).

2. `compute_viscosity()` builds $\mathbf{D}(\mathbf{v}^{(k)})$ from those face values (Eq. (36)), reduces it to $\dot{\gamma}^{(k)}$ (Eq. (38)), and then to $\eta_a^{(k)}$ (Eq. (44)); this *is* Eq. (42):

   Listing 5: `compute_viscosity()`: $\mathbf{v}^{(k)} \to \mathbf{D} \to \dot{\gamma} \to \eta_a^{(k)}$, held fixed through steps 3–4.

```
double Dxx=(uxe[i]-uxw[i])/dx[i], Dyy=(uyn[i]-uys[i])/dy[i];
double dxvy=(uye[i]-uyw[i])/dx[i], dyvx=(uxn[i]-uxs[i])/dy[i];
double Dxy=0.5*(dyvx+dxvy);
double DdotD=Dxx*Dxx+Dyy*Dyy+2.0*Dxy*Dxy;    // D:D
double gamma_dot=sqrt(2.0*DdotD);            // gamma_dot = sqrt(2*D:D), Eq. (35)
double Deff=sqrt(gamma_dot*gamma_dot+POWERLAW_EPSILON*POWERLAW_EPSILON);
eta[i]=POWERLAW_K*pow(Deff, POWERLAW_N-1.0);
```

3. `predict_velocity_nonnewtonian()` uses $\mathbf{v}^{(k)}$ for the advection terms and the just-computed, now-frozen $\eta_a^{(k)}$ for the full stress-divergence term

$$\big[\nabla\cdot(2\eta_a\mathbf{D})\big]_x \approx \frac{\tau_{xx}^{e} - \tau_{xx}^{w}}{\Delta x} + \frac{\tau_{xy}^{n} - \tau_{xy}^{s}}{\Delta y}, \tag{45}$$
$$\tau_{xx} = 2\eta_a\,\partial_x v_x, \qquad \tau_{xy} = \eta_a\big(\partial_y v_x + \partial_x v_y\big),$$

   with face viscosities from a plain average of the two neighbouring cell centers, $\eta_a^{e} = \frac{1}{2}(\eta_a^{c} + \eta_a^{E})$, and tangential derivatives (e.g. $\partial_y v_x$ at the east face) from averaging the two cell-centered tangential gradients sharing that face, since this grid has no diagonal-neighbour stencil. The result is the predicted field $\mathbf{v}^{*}$, the momentum half of Eq. (43), with $\eta_a$ still frozen.

4. `compute_b_src()` and `solve_pressure_sor()` measure $\nabla\cdot\mathbf{v}^{*}$ and solve Eq. (34) for the correction $\phi$, exactly the Newtonian projection step of Section 6, oblivious to the fact that $\eta_a$ ever varied (Section 7.3).

5. `correct_velocity()` applies Eq. (33) to produce $\mathbf{v}^{(k+1)}$, closing the pressure half of Eq. (43).

6. The outer `do...while` checks $\|\mathbf{v}^{(k+1)} - \mathbf{v}^{(k)}\|$ against the tolerance. If it has not converged, $\mathbf{v}^{(k+1)}$ becomes the new $\mathbf{v}^{(k)}$ and step 1 runs again, so $\eta_a$ is rebuilt from a slightly different velocity every single pass, the same lagging of Eqs. (42)–(43), not a one-off calculation.

# 8 Models overview and Results

Instead of having one program, that solves directly the shear-thinning problem, five separate codes are presented, since it describes how the final goal was reached. The same grid ($N_x = 100$, $N_y = 20$) is shared by all five cases, so every result can be directly compared to the next.

## 8.1 Files and milestones

- **finite_vol_1_baseline.c**: the baseline, in which a periodic torus domain is set, and velocity divergency is assumed without confirmation. It is not actually a milestone, but it is the model for the development of the project.
- **finite_vol_2_duct.c**: real boundary conditions are added, keeping the Newtonian fluid (constant viscosity), and the pressure is solved to reach convergence in every step.
- **finite_vol_3_syringe.c**: The syringe geometry is applied and the mass-conservation failure is highlighted.
- **finite_vol_4_syringe_SIMPLE.c**: Mass conservation is restored by applying the SIMPLE method, which makes the velocity update based on a pressure correction needed to fulfill the conservation of mass .
- **finite_vol_5_duct_SIMPLE_shearthinning.c**: the plain duct geometry is used again, the SIMPLE method is kept, and the constant viscosity is replaced by the regularized power-law viscosity.
- **finite_vol_6_syringe_SIMPLE_shearthinning.c**: the syringe geometry, the SIMPLE method, and the power-law viscosity are combined here, to finally reach the goal of simulating a Non-Newtonian flow through a reduced area.

The legacy and SIMPLE method comparison for the first three cases is tabulated in Table 1 (Section 6.3). The equivalent comparison for the two shear-thinning files is shown directly in the flow-rate profiles (Figure 4).

## 8.2 Velocity profiles

In Figure 2, $v_x$ is shown across the channel height at $x = 90$, with Newtonian compared against power-law for each geometry. In the plain duct, the contrast predicted by the power-law model is very clear. The Newtonian fluid has the expected parabolic profile, while the shear-thinning profile is visibly flatter in the core, and steeper right at the walls, where $\eta_a$ is low. In the syringe throat, the same trend is observed, but weaker there, because the throat is already narrow. So a more uniform shear rate is experienced across the whole cross-section, leaving less room for the contrast between the core and the walls. The primary observation to be made here is that the velocity of the non-Newtonian fluid is higher in proximity to the wall and lower in the core than that of the Newtonian fluid, reaching a smaller maximum velocity. Consequently, fluids with high shear-thinning tendencies will have a flatter core velocity profile and reach lower velocities than a comparable Newtonian fluid. Figure 3 shows the full 2D velocity field for the syringe, power-law case, `finite_vol_6_syringe_SIMPLE_shearthinning.c`.

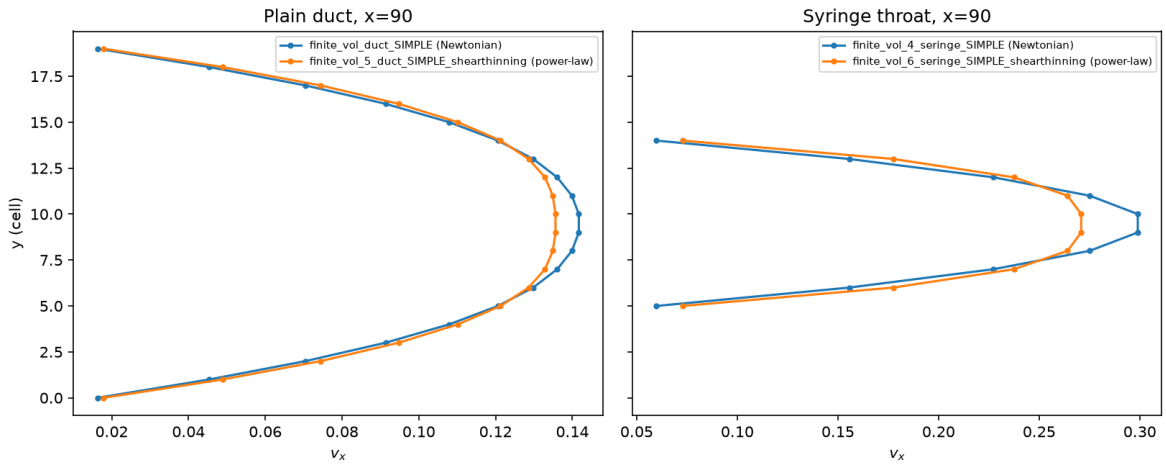


Figure 2: $v_x$ across the channel height at $x = 90$, Newtonian versus power-law, for the plain duct (left) and the syringe throat (right).

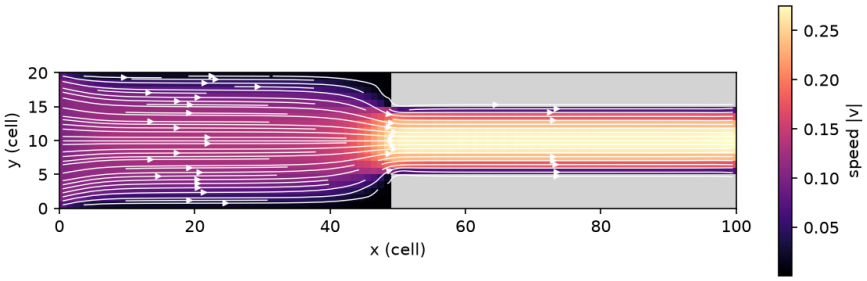


Figure 3: Speed $|v|$ and streamlines for the syringe, power-law case.

### 8.3 Flow-rate profiles

In Figure 4, $Q(x)$ from all four cases is plotted on the same axes. Three of the four, the ones in which the projection method is actually used, are kept flat at the mass-conserving value the whole way down the duct, with the constriction included. The fourth, plain-duct Newtonian, in which the legacy pressure equation is left untouched, is collapsed within the first 20 cells and is never recovered, exactly the failure that was documented quantitatively in Section 5 and Table 1. When all four are seen together like this, the point made in Section 6 is shown visually: it is the pressure treatment, not the geometry or the rheology, that is found to decide whether mass is conserved.

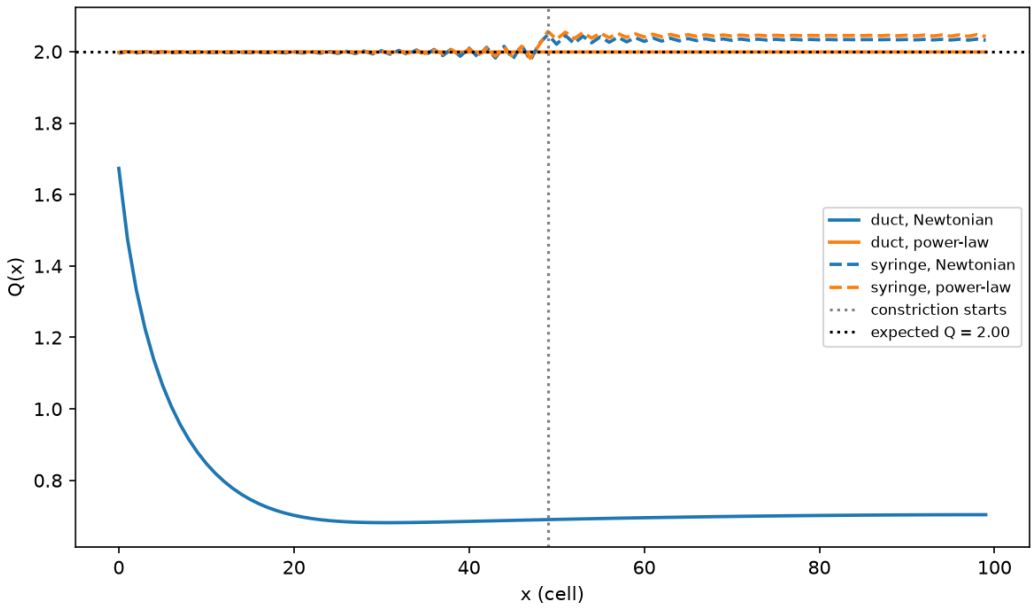


Figure 4: $Q(x)$ for all four milestone cases.

### 8.4 Apparent viscosity

In Figure 5, the apparent-viscosity profile across the channel height, at $x = 90$, is compared between `finite_vol_2_duct.c` (Newtonian) and `finite_vol_5_duct_SIMPLE_shearthinning.c` (power law, $K = 0.5$, $n = 0.6$). This shape is the one that was expected: because $\eta_a$ is made a strictly decreasing function of $\dot{\gamma}$ by Eq. (35) once $n = 0.6 < 1$ is chosen, and because $\dot{\gamma}$ is highest at the walls and lowest at the centerline (Section 7), $\eta_a$ is found to be lowest at the walls and highest at the centerline, just as shown by the power-law curve.

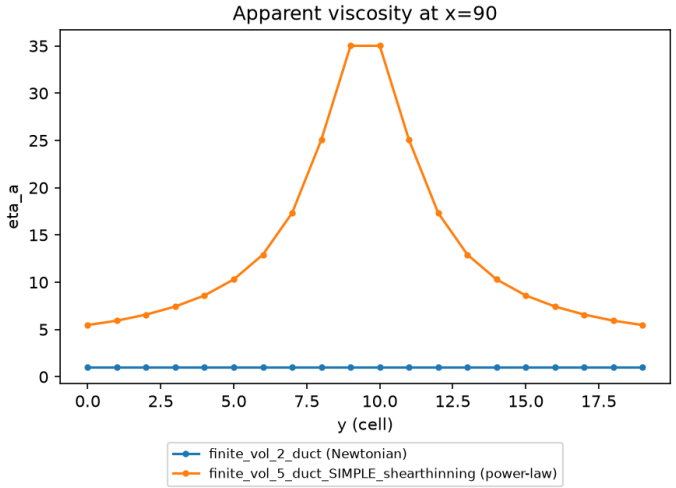


Figure 5: Apparent viscosity $\eta_a$ across the channel height at $x = 90$, Newtonian versus power-law.

# 9 Conclusion

In this project, a shear-thinning flow simulation was built up from the Navier–Stokes equations to a power-law viscosity model coupled with the SIMPLE/projection method, worked through as five milestones. The finite-volume machinery was established by the base case, a real physical shape was given to the domain by the boundary conditions, and a mass-conservation failure was exposed by the syringe geometry. That failure was found to already be present on a plain duct, and got worse by the syringe geometry. It was then then fixed by the SIMPLE method. With a valid Newtonian model in place, the constant viscosity was finally replaced by a regularized, shear-rate-dependent power law, which shows the "thinning" characteristic expected and keep relative good mass conservation, once combined with the SIMPLE method. A couple of things are still left to be improved:

- The power-law parameters $K = 0.5$ and $n = 0.6$ are placeholders, and they still need to be real values to be comparable with real experiments..
- More advances models, such Carreau or Herschel–Bulkley could be adopted instead of the power law used here.
- The SIMPLE method applied did not conserved the flux with great precision. Improvements could done, and more complex geometries could lead to even worse results.

# References


[1] Korzec, *Computational Fluid Dynamics*, Lecture Notes, Chapter 3. Course material for this project; the source of the finite-volume method, the discretized momentum update (Eqs. 3.53–3.59), and the pressure-Poisson/Jacobi treatment used in Sections 2–4.

[2] Yadav, *Non-Newtonian Fluids for Industrial Applications*, 2026, Chapter 2, Section 2.3.5 (“Power Law Fluids”). Source of the rate-of-strain tensor, the power-law constitutive relation, and the non-Newtonian pressure-Poisson derivation used in Section 7.

[3] Baheri, A., Navaserarab, A., “Application The Pressure Correction Method for Incompressible Viscous Flow,” *International Journal of Scientific Engineering and Applied Science (IJSEAS)*, 2(2):93–97, 2016. https://ijseas.com/volume2/v2i2/ijseas20160210.pdf. Public reference describing the SIMPLE (pressure-correction) method used in Section 6.